

Spaceship Titanic - Group 2

1st Anant Sahoo
Department of EECS
University of Tennessee, Knoxville
Knoxville, United States
asahoo@vols.utk.edu

2nd Deep Patel
Department of EECS
University of Tennessee, Knoxville
Knoxville, United States
dpate125@vols.utk.edu

3rd Sulaiman Mohyuddin
Department of EECS
University of Tennessee, Knoxville
Knoxville, United States
smohyud1@vols.utk.edu

4th William Sessoms
Department of EECS
University of Tennessee, Knoxville
Knoxville, United States
wsessoms@vols.utk.edu

Abstract—This project focuses on improving the baseline model for the Kaggle "Spaceship Titanic" challenge [1]. The original tutorial used a Random Forest built with TensorFlow Decision Forests, which gave a baseline accuracy of 78.653%. Our goal was to push this score higher without changing the type of model, and instead focus on better data handling and practical preprocessing steps. We moved the implementation to Scikit-learn to gain more control over the pipeline, applied one-hot encoding and feature scaling, and used imputation methods to handle missing values instead of dropping large parts of the dataset. We also introduced stratified splitting, set a fixed random seed for consistent results, and tuned hyperparameters using `RandomizedSearchCV`. With these changes, our final submission reached 79.752% accuracy. While the improvement is modest, it reflects how careful preprocessing, consistent handling of missing data, and small adjustments can meaningfully improve performance on tabular datasets.

Index Terms—machine learning, random forest, data preprocessing, imputation, tabular data, hyperparameter tuning

I. INTRODUCTION

In this science fiction scenario from Kaggle, the year is 2912, and humanity has achieved interstellar travel. Recently, over 13,000 passengers have boarded the *Spaceship Titanic*, a grand vessel set out to explore newly habitable exoplanets orbiting nearby stars. The voyage reached an abrupt ending after colliding with a spacetime anomaly, resulting in almost half of the passengers being transported to an alternate dimension. To help maximize the chances of retrieving the lost passengers, a machine learning model is utilized to predict which passengers were transported to the alternate dimension.

The passenger dataset consists of a training and testing split. The training set includes about two-thirds (8693 passengers), while the testing set includes the other third (4277). Each passenger has a unique passenger identification along with other information, including but not limited to their home planet, whether they were in cryo sleep for the voyage, cabin number, destination, age, and whether the passenger got transported to the other dimension.

Kaggle's baseline model uses a Random Forest with TensorFlow Decision Forests. The rationale being that decision

forests provide a strong baseline when working with tabular data, often beating neural networks without heavy experimentation. The model was trained using TensorFlow's default hyperparameters, and the Out-of-Bag (OOB) and validation accuracy were evaluated. For the submission, the model outputs a CSV file where each row includes a passenger ID and their predicted transported value, which is then checked against the test set's truth labels. The purpose of this paper is to improve the baseline model's accuracy of 78.653% by further exploring data preprocessing steps.

II. IMPROVEMENT IDEAS AND EXPERIMENTS

A. Sci-Kit Learn Models

The TensorFlow method of doing random forests does not allow for changes to the pipeline as easily as using Scikit-learn would. Because of this, we have changed the random forest implementation from TensorFlow to Scikit-learn. Using the Scikit-learn method of implementing random forests also allowed us to implement other Scikit-learn machine learning techniques, like the `ColumnTransformer`. We used the column transformer to transform some of the columns to allow the model to better use the data. In this `ColumnTransformer`, we applied the `OneHotEncoder` to the `HomePlanet`, `Destination`, `Deck`, and `Side` columns. In addition, we also used `StandardScaler` and applied it to the `Age`, `RoomService`, `FoodCourt`, `ShoppingMall`, `Spa`, and `VRDeck` columns. The `OneHotEncoder` allowed us to use categorical data that we may not have been able to otherwise. Meanwhile, the `StandardScaler` allowed us to ensure that the model is no longer sensitive to the magnitude of values and instead focuses on the variation in the values.

B. Imputed Samples with Null Values

Several features of the dataset had a significant amount of invalid or null values. This can be problematic because machine learning algorithms are not guaranteed to handle missing values perfectly, and removing the null values can

result in detrimental data or performance loss. For numeric features such as Age and RoomService, we preserved the overall scale and distribution of these variables by replacing missing values with the arithmetic mean. We applied mode imputation for categorical features like Destination, Side, Deck, and HomePlanet by replacing values with the most frequently occurring category. This approach ensures data remains consistent while preventing data or model training loss.

C. Best Practices

We had some concerns about the baseline model's consistency, reliability, and model quality. During the baseline submission process, we observed that the accuracy score did not remain consistent, differing by around a hundredth each submission. Upon further investigation, we determined that the model did not have a predefined seed for random number generation. We addressed this by adding a seed value, thereby guaranteeing deterministic results from our model. Another problem we addressed was how the dataset was split. Instead of a manual split, we used Sci-Kit learn's stratified train-test-split method on the Transport variable. Stratification ensures that the class proportions in the training and test sets remain consistent with the original dataset, preventing skewed splits that could bias the model or inflate accuracy. Finally, we decided to drop the Cabin-num field entirely because over 200 entries were missing, and imputation provides little benefit for something that offers negligible value for prediction anyway.

III. RESULTS AND DISCUSSION

The original baseline model that we created from following the provided tutorial resulted in a final submission accuracy of 78.653%, which we improved to 79.752% by making the stated changes. This is an increase of 1.099% without changing the type of model used, simply from cleaning up the data and properly tuning hyperparameters. Our RandomizedSearchCV was very successful in finding proper hyperparameters, as tuning by hand was not very efficient, and left a lot of room for error. Our final submission with the hyperparameters tuned from our RandomizedSearchCV was 0.094% better than the hyperparameters resulting from manual tuning. The main trend we noticed was that imputing data was very important. If we were to simply drop the data instead, we would have lost over 1000 rows in this dataset, hurting our ability to train and potentially missing out on patterns in the data. Considering that the top model submitted to this competition had ~82% accuracy, we believe our improvement to 79.752% accuracy to be substantial.

IV. CHALLENGES AND LESSONS LEARNED

While this part does not deal with the model itself, we did run into a few issues during the submission process. At first,

our submission CSV used 1 and 0 for the Transported field. We assumed this was fine since many programming frameworks treat 1 as true and 0 as false, and in most cases they behave the same as Boolean values. After a few failed submissions that came back with an accuracy of 0, we went back to the project overview and realized the Transported field had to be an explicit Boolean. This was a small oversight on our end, but it did cost us some time and was a bit frustrating.

Handling missing data was another challenge. We had to work around values that simply were not there. For the submission format, the row count had to stay consistent, and there was no limit on column count, meaning any missing entries had to be filled in with something reasonable. We had to make judgment calls on what those values should be. Since CabinNum had the most missing values and did not seem very useful for the model, we decided to remove it. The harder part was choosing replacement values for the other missing entries. Overall, this taught us that real-world data is rarely clean, and sometimes you have to rely on trial-and-error or reasonable assumptions to make the dataset workable for a model.

TEAM CONTRIBUTION STATEMENT

- Anant Sahoo: Baseline Model execution and commenting; kaggle submission for Baseline and Improved model; Group Leader and Coordinator; Idea generation for Improved; Evaluation and Analysis; Report Writing and Editing; Organizing submissions.
- Deep Patel: Data handling and feature work for the Improved model; Report writing and editing; Idea generation for Improved; Evaluation and Analysis.
- Sulaiman Mohyuddin: Idea generation for an Improved model; Report writing and editing; Data Handling.
- William Sessoms: Coding and Experimentation with Improved Model; Hyperparameter Tuning; Report Writing and Editing.

REFERENCES

- [1] Kaggle, "Spaceship Titanic," *Kaggle Competitions*. [Online]. Available: <https://www.kaggle.com/competitions/spaceship-titanic/overview>. Accessed: Dec. 1, 2025.